

Chapter 25: Process and Project Metrics

Software Engineering

BTech Computer Science and Engineering

Duration: 1 Hour



Process and Project Metrics

Measuring the Engine of Software Engineering

The Shift in Focus:

Moving from Product Metrics (measuring the software itself) to Process & Project Metrics (measuring the environment that creates it).

The Ultimate Goal:

To improve, predict, and control our organization's software engineering capabilities.

Key Questions We Will Answer:



How productive is our engineering team?



How accurate are our timeline and cost estimates?



How does our software quality stack up against industry benchmarks?

Lecture Objectives

What We Will Cover Today

By the end of this lecture, you will be able to:



Distinguish between process metrics and project metrics and explain their specific use cases.



Apply size-oriented (LOC) and function-oriented (FP) metrics to measure team productivity and output quality.



Understand the application of object-oriented and use-case-oriented metrics in modern development.



Measure software quality utilizing Defect Removal Efficiency (DRE) and other key quality indicators.



Describe how to successfully integrate metrics within the software process and establish a sustainable metrics program.

Metrics in the Process and Project Domains

Measuring the Factory vs. Measuring the Product



Process Metrics: The How



- Measure the overall software development lifecycle.
- Evaluate methodology used across the entire organization.
- Focus on long-term efficiency and improvement.



Project Metrics: The What



- Measure specific characteristics of a single project.
- Track timeline, budget, and scope.
- Assess progress and immediate deliverable quality.



The Interconnection:



Better processes lead to more predictable and successful projects.

Process Metrics

Understanding and Improving How We Work

Definition & Purpose



Metrics that measure the software development process itself, completely independent of the final product.



To understand, evaluate, and ultimately improve the organization's software process over time.

Key Examples:



Defect Removal Efficiency (DRE): How effectively are we catching bugs across different phases before release?



Review Effectiveness: How many defects are found per hour of peer review?



Rework Effort: What percentage of our total effort is wasted fixing mistakes?



Process Compliance: What percentage of our projects are actually following our defined organizational processes?

Software Process Improvement

The IDEAL Cycle

To systematically improve our software process based on the metrics we collect, we use the IDEAL framework:



Project Metrics

Estimating, Tracking, and Controlling

Definition & Purpose



Definition: Metrics that measure the specific characteristics and daily realities of an active project.



Purpose: To accurately estimate future work, track ongoing progress, and control deviations before they become fatal.

Key Examples



Schedule Variance: Actual effort vs. planned effort (Are we behind schedule?).



Cost Variance: Actual cost vs. planned cost (Are we over budget?).



Productivity: Output measured in Lines of Code (LOC) per person-month or Function Points (FP) per person-month.

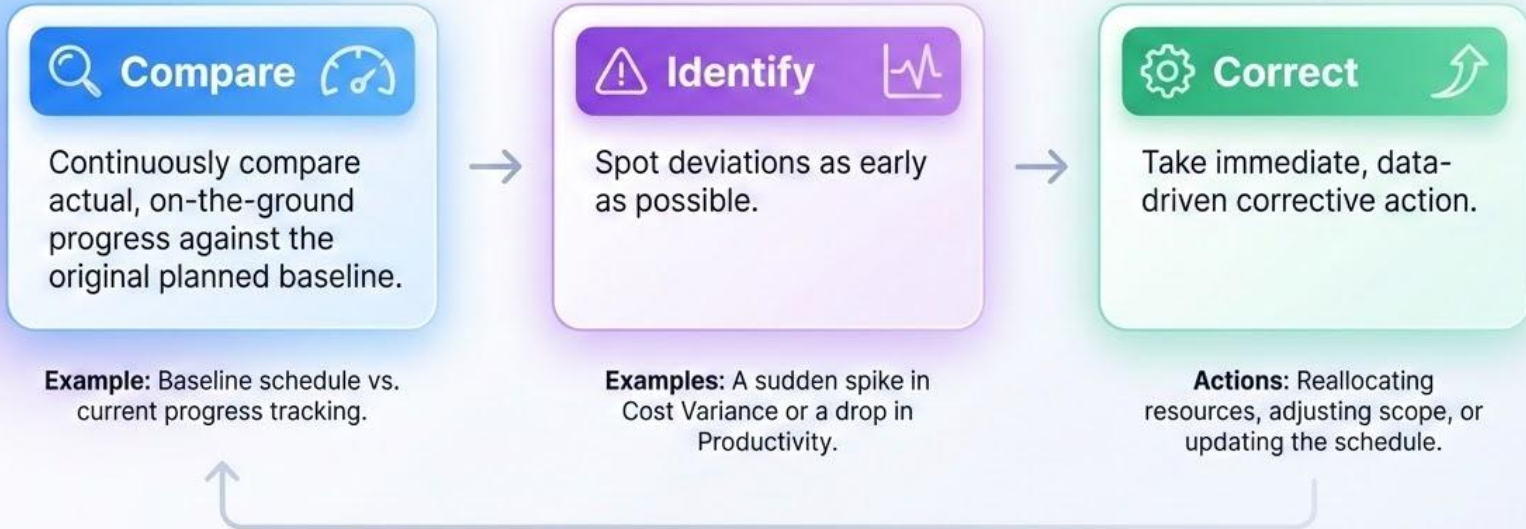


Defect Density: The number of defects found per KLOC (Thousand Lines of Code) or per Function Point.

The Use of Project Tracking

Reacting to the Data

Project metrics are the dashboard instruments used to safely navigate a software build.



Measuring the Code itself

Size-Oriented Metrics: The Lines of Code (LOC) Approach



Definition

Metrics that are normalized per thousand lines of code (KLOC).

Common Metrics

-  **Productivity:** KLOC / person-month
-  **Quality:** Defects / KLOC
-  **Cost:** Project cost / KLOC
-  **Documentation:** Pages of documentation / KLOC

Advantages

- Very simple to understand
- Easy to compute automatically
- Widely used historically

Disadvantages

- Highly language-dependent (penalizes concise languages like Python)
- Very difficult to estimate early in the project before code is written

Function-Oriented Metrics

The Function Point (FP) Approach

Definition: Metrics normalized by the functionality delivered to the user, known as a Function Point (FP).

Common Metrics

-  **Productivity:** FP / person-month
-  **Quality:** Defects / FP
-  **Cost:** Project cost / FP
-  **Documentation:** Pages of documentation / FP

Advantages

- ✓ Language-independent
- ✓ Focuses on the actual functionality delivered
- ✓ Can be estimated directly from the requirements phase

Disadvantages

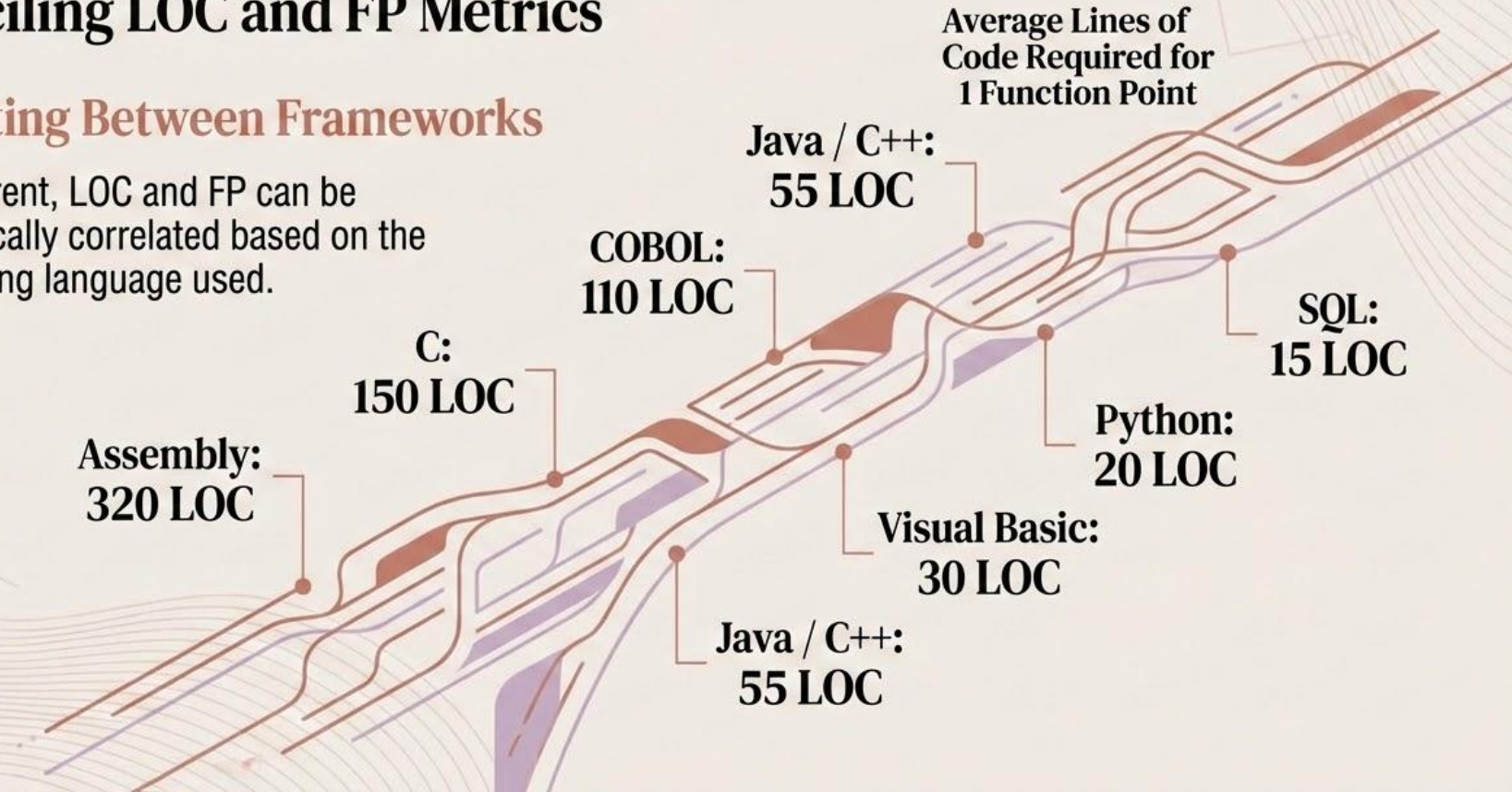
- ✗ Requires highly trained experts to count accurately
- ✗ Involves subjective interpretation

Bridging the Gap

Reconciling LOC and FP Metrics

Translating Between Frameworks

While different, LOC and FP can be mathematically correlated based on the programming language used.



Modern Paradigms

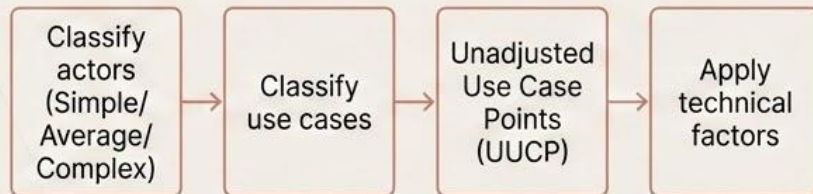
Object-Oriented & Use-Case Metrics: Measuring Modern Architecture

Project-Level OO Metrics (Lorenz & Kidd)

- **Key Classes:** Count of core domain classes (indicates system size).
- **Support Classes:** Count of UI/DB classes (indicates infrastructure size).
- **Average Operations per Class:** If >20 , suggests high complexity.

Use-Case-Oriented Metrics

- **Number of Use Cases:** A rough, immediate size indicator.
- **Use Case Points (UCP):** A weighted calculation similar to FP.
- Calculation:



WebApp Project Metrics

Specific Metrics for Web Development

Content

- Number of static pages, dynamic pages, images, videos
- **Purpose:** Size estimation

Functionality

- Number of user functions, backend transactions
- **Purpose:** Complexity assessment

Navigation

- Number of links, distinct navigation paths
- **Purpose:** Effort estimation

Performance

- Expected load time, peak concurrent users
- **Purpose:** Infrastructure planning

Metrics for Software Quality

Quantifying "Good" Software

The Challenge

"Quality" is often subjective. We must translate subjective feelings into objective, measurable data.

The Five Dimensions of Quality

Quality Dimension

- Correctness
- Maintainability
- Integrity
- Reliability
- Usability

Example Metric

- Defects per KLOC (Post-release)
- Mean Time to Change (MTTC)
- Number of security vulnerabilities
- Mean Time To Failure (MTTF)
- User satisfaction survey scores

Evaluating the Codebase

Correctness, Maintainability, and Integrity

- **Correctness
(Defect Density)**

Formula: Total Defects / Size
(KLOC or FP)

Industry Benchmarks:

- Excellent: < 1 defect per KLOC
- Good: 1-2 defects per KLOC
- Poor: > 5 defects per KLOC

- **Maintainability
(MTTC)**

Mean Time to Change
(MTTC): The average time
required to implement a
change request. A lower
MTTC indicates a highly
maintainable, clean
codebase.

- **Integrity
(Security)**

Formula: Threat (Probability
of an attack) × Potential
Loss.

Defect Removal Efficiency (DRE)

Catching Bugs Before Release

The Definition

The percentage of all defects found before the software is released to the customer.

The Formula

$$DRE = \frac{E}{E + D}$$

E = Defects found before release (by the team).

D = Defects found after release (by the customer).

Example

90 defects found during development. **10** found by users after release.

$$DRE = \frac{90}{90 + 10} = 0.90 \rightarrow \mathbf{90\%}$$

Interpreting DRE Scores

What Does the Math Tell Us?

Interpreting the Final Score

- **DRE = 100%**: Perfect (No post-release defects). Highly rare.
- **DRE > 95%**: Excellent quality. The process is working as intended.
- **DRE < 80%**: Poor quality. Immediate improvement needed in testing protocols.

Phase-Level DRE

- DRE should be tracked for each phase (Requirements, Design, Coding, Testing).
- If a phase has a low DRE, it indicates a critical need for better peer reviews or testing procedures in that specific phase.

The Great Debate: Integrating Metrics within the Software Process

Arguments For and Against Measurement

The Core Argument For Metrics

“What gets measured gets managed.”

- Data-driven decisions are inherently better than gut-feeling guesses.
- Metrics are the only way to prove continuous improvement.

Common Cultural Objections & Counterarguments

- ✗ “Metrics take too much time to collect.” → Automate collection to minimize overhead.
- ✗ “People will game the system to look good.” → Focus exclusively on process metrics, never on individual developer performance.
- ✗ “Management will weaponize these numbers against us.” → Start simple, build trust, and grow the program gradually.

Establishing a Baseline

You Cannot Improve What You Have Not Measured

What is a Baseline?

- Historical data collected from past projects that serves as a fixed reference point.

Why do we need it?

- It allows us to compare the current project's performance against historical reality.
- It drastically improves future estimation accuracy.
- It proves whether a process change is actually working.

Core Metrics to Baseline



Productivity: (e.g., Function Points per person-month)



Quality: Defect density (defects per KLOC)



Testing Efficiency: Defect Removal Efficiency (DRE) by phase



Accuracy: Schedule and Cost variance (Actual vs. Planned)

Collection, Computation, and Evaluation

Running the Metrics Program

1. Collection (Make it Invisible)



- Automate wherever possible (pull data directly from Git, Jira, CI/CD tools).
- Integrate data collection into the daily workflow; it should never be an afterthought.

2. Computation (Make it Consistent)



- Use strictly consistent formulas across all teams.
- Document calculation methods clearly so the math is transparent.
- Validate the raw data for accuracy before running the numbers.

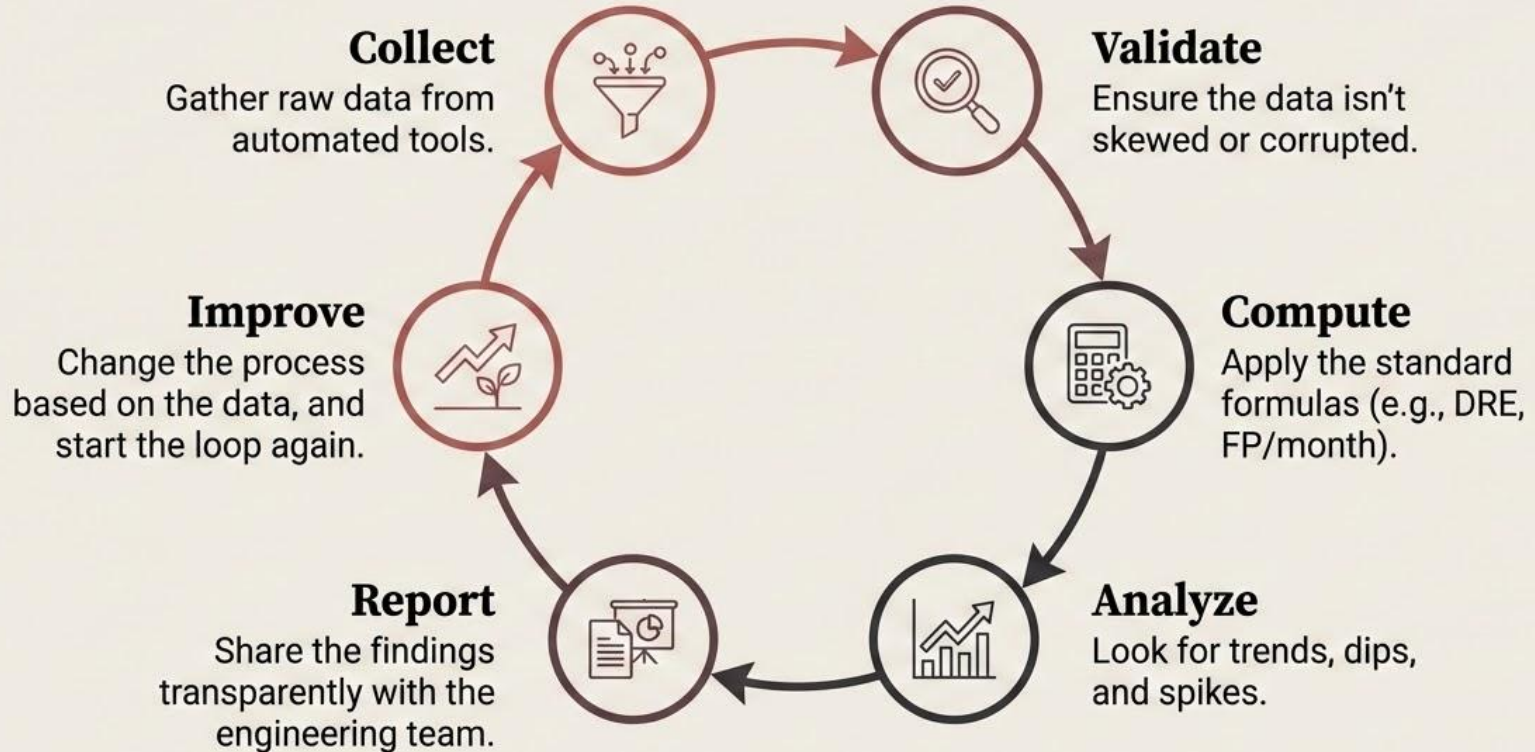
3. Evaluation (Make it Actionable)



- Compare the newly computed metrics against the established baseline.
- Identify outliers and investigate the root causes.
- Communicate the results openly to all stakeholders to drive actual improvement.

The Metrics Flow

From Raw Data to Process Improvement



Metrics for Small Organizations

Scaling Measurement for Lean Teams

The Reality for Small Teams



- **Limited Resources:** Cannot afford dedicated metrics analysts.
- **Lack of Data:** Fewer past projects mean a weak historical baseline.
- **Informal Processes:** Rigid, heavy metrics frameworks will crush agility.

The Practical Approach



- **Start Small:** Choose exactly 3 to 5 high-value metrics. No more.
- **Keep It Simple:** Rely strictly on automated collection.
- **Focus on Process:** Measure the workflow, never individual performance.
- **Use Lightweight Tools:** Spreadsheets and basic Jira/GitHub dashboards are enough.

The Small Team Starter Kit

High-Value, Low-Overhead Metrics



Project Size

(Count Use Case Points or Function Points).



Estimation Accuracy

Actual vs. Planned Effort (Schedule/Cost Variance).



Pre-Release Quality

Total defects found during testing.



Post-Release Quality (DRE)

Defects found after release (Used to calculate Defect Removal Efficiency).

Establishing a Software Metrics Program

Steps to Lasting Success

1. **Identify Business Goals:** What exact problem are we trying to solve? (e.g., "Reduce customer complaints").
2. **Define Metrics (The GQM Framework):** Map the Goal to a specific Question, and answer it with a Metric.
3. **Select Carefully:** Every single metric must have a clear, justified purpose.
4. **Establish Collection:** Automate data gathering into the CI/CD pipeline.
5. **Create a Baseline:** Aggregate current and past project data.
6. **Analyze and Report:** Provide transparent, regular reports to the team.
7. **Act:** Never collect data just to collect it; use findings to drive actual process changes.

Protecting the Culture

Critical Success Factors

Making the Program Stick

- ◆ **Executive Management Support:** Leaders must champion the effort and fund the tooling.
- ◆ **Absolute Clarity in Communication:** The team must understand that metrics are tools for learning, never weapons for blame.
- ◆ **Automated Collection:** Manual data entry will be abandoned under deadline pressure.
- ◆ **Regular Review and Refinement:** If a metric stops providing actionable value, delete it.

Conclusion & Key Takeaways

The Dual Lenses of Metrics

Process vs. Project:

- ◆ **Process Metrics:** Measure the overall development cycle to drive continuous organizational improvement (via the IDEAL model).
- ◆ **Project Metrics:** Measure the specific, current project to estimate, track, and control the daily work.

Traditional Estimation:

- ◆ **Size-Oriented (LOC):** Simple to calculate, but highly language-dependent.
- ◆ **Function-Oriented (FP):** Language-independent and excellent for early estimation based purely on delivered business value.

Key Takeaways

Modern Frameworks and the Golden Metric

Modern Measurement:

- ◆ **Object-Oriented Metrics:**
Focus on counting key classes, support classes, and average operations.
- ◆ **Use-Case-Oriented Metrics:**
Utilize Use Case Points (UCP) for highly accurate, user-behavior-driven size estimation.

Software Quality Metrics:

- ◆ **Defect Density:** Bugs per KLOC.
- ◆ **Maintainability (MTTC):** Mean Time to Change.
- ◆ **Defect Removal Efficiency (DRE):** The most critical metric—the exact percentage of defects your team successfully caught before release.

Key Takeaways

Making Metrics Stick

Integrating Metrics:

- ◆ Always establish a historical baseline first.
- ◆ Automate collection to eliminate developer overhead.
- ◆ Use the data exclusively for process improvement, never for individual punishment.

Small Organizations:

- ◆ **Start Small:** Choose only 3-5 high-value metrics.
- ◆ **Grow Gradually:** A mature metrics program requires clear goals (GQM), careful selection, automated collection, and decisive action based on the analysis.

The Final Word

From Data to Wisdom

“ Metrics are not about judgment. They are about visibility. A metric that shows poor productivity is not a failure—it’s an opportunity to ask why and improve. The best metrics programs turn data into wisdom. ”